UNIVERSITÉ DE JIJEL
Faculté des Sciences Exactes et Informatique
Département d'Informatique

Rapport de Travaux Pratiques

Module : Recherche Intelligente et Coopération

TP : Cartographie d'une Fonction Mystère

Niveau : MASTER 1 — Intelligence Artificielle

Année Universitaire : 2025–2026

Réalisé par :

KRIKET Nassira

Encadré par :

Dr. ZENNIR Nadjib

Résumé

Ce rapport présente l'implémentation complète d'un **Perceptron Multicouche (MLP)** réalisée entièrement en **Python** avec la bibliothèque **NumPy**, sans recours à aucun framework de haut niveau tel que PyTorch ou TensorFlow.

L'objectif principal est d'entraîner ce réseau à apprendre et à reconstruire une surface mathématique complexe :

$$f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right) + 0.5 \cdot \cos(2x + 2y)$$

Le projet est structuré en quatre étapes progressives : la **génération et normalisation des données**, la **conception de l'architecture MLP**, l'implémentation manuelle de la **rétropropagation du gradient**, et **l'entraînement** avec visualisation des résultats. Des expériences complémentaires analysent l'impact de l'activation linéaire, de la capacité du réseau et du momentum.

Abstract

This report presents the complete implementation of a **Multilayer Perceptron (MLP)** built entirely in Python using only NumPy, without any deep learning framework. The network is trained to approximate the complex mathematical surface $f(x, y) = \sin(\sqrt{x^2 + y^2}) + 0.5\cos(2x + 2y)$.

Mots-clés : Perceptron Multicouche, Rétropropagation, NumPy, SGD, Momentum, Apprentissage Profond.

Contents

1 Introduction et Contexte **3**
1.1 Contexte général . . . 3
1.2 Objectif du TP . . . 3
1.3 Fonction cible . . . 3

2 Génération et Préparation des Données **4**
2.1 Génération du dataset . . . 4
2.2 Visualisation de la surface réelle . . . 4
2.3 Normalisation des données . . . 5

3 Architecture du Réseau MLP **6**
3.1 Structure du réseau . . . 6
3.2 Initialisation des poids — He . . . 6
3.3 Fonctions d'activation . . . 6
3.4 Fonction de perte — MSE . . . 6

4 Rétropropagation du Gradient **8**
4.1 Principe . . . 8
4.2 Couche de sortie (activation linéaire) . . . 8
4.3 Couches cachées (activation ReLU) . . . 8
4.4 Mise à jour des poids — SGD . . . 9

5 Entraînement et Résultats **10**
5.1 Hyperparamètres . . . 10
5.2 Courbe de loss . . . 10
5.3 Comparaison Ground Truth vs Prédictions . . . 11

6 Questions de Réflexion **12**
6.1 Q1 — Impact de l'activation linéaire . . . 12
6.2 Q2 — Capacité du réseau (petit réseau) . . . 12
6.3 Q3 — SGD avec Momentum . . . 13

7 Conclusion **15**

Annexes **16**

1. Introduction et Contexte

1.1 Contexte général

L'apprentissage profond est au cœur des avancées modernes de l'intelligence artificielle. Le **Perceptron Multicouche (MLP)** constitue la brique fondamentale de ces systèmes. Ce TP propose de l'implémenter entièrement *from scratch* avec NumPy, afin de comprendre en profondeur chaque mécanisme interne.

Définition : Perceptron Multicouche (MLP)

Un MLP est un réseau de neurones feedforward composé d'une couche d'entrée, de couches cachées et d'une couche de sortie. Chaque couche est entièrement connectée à la suivante. Il constitue la base de l'apprentissage profond moderne.

1.2 Objectif du TP

Implémenter un MLP capable d'apprendre les points suivants :

- Propagation avant (*forward propagation*)
- Calcul de la perte MSE (*Mean Squared Error*)
- Rétropropagation du gradient (*backpropagation*) vectorisée
- Optimisation par descente de gradient stochastique (*mini-batch SGD*)

1.3 Fonction cible

$$\boxed{f(x, y) = \sin\left(\sqrt{x^2 + y^2}\right) + 0.5 \cdot \cos(2x + 2y)} \tag{1.1}$$

Cette fonction présente des oscillations non-linéaires complexes dans les deux dimensions, ce qui constitue un défi intéressant pour le réseau.

2. Génération et Préparation des Données

2.1 Génération du dataset

Un ensemble de **2000 points** (x, y) est généré aléatoirement et uniformément dans $[-5, 5]^2$ avec `seed=42` pour la reproductibilité.

```
def mystery_function(x, y):
    return np.sin(np.sqrt(x**2 + y**2)) + 0.5 * np.cos(2*x + 2*y)

def generate_data(n_points=2000, x_min=-5, x_max=5, seed=42):
    np.random.seed(seed)
    x = np.random.uniform(x_min, x_max, n_points)
    y = np.random.uniform(x_min, x_max, n_points)
    z = mystery_function(x, y)
    return x, y, z
```

Listing 2.1: Génération des données — generate.py

2.2 Visualisation de la surface réelle

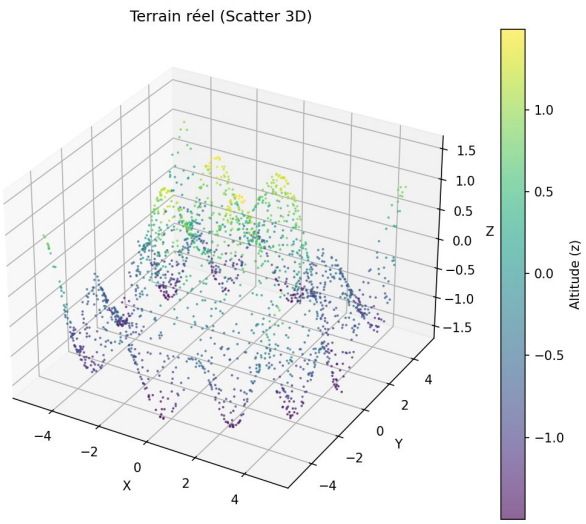


Figure 2.1: Scatter 3D de la fonction mystère $f(x, y)$ sur 2000 points aléatoires

2.3 Normalisation des données

Une **normalisation z-score** est appliquée à chaque variable pour faciliter la convergence :

$$x_{norm} = \frac{x - \mu_x}{\sigma_x} \tag{2.1}$$

```python
def normalize(data):
    mean = np.mean(data)
    std  = np.std(data)
    return (data - mean) / std, mean, std
```

Listing 2.2: Normalisation z-score — normalize.py

Table 2.1: Paramètres de normalisation obtenus

Variable	**Moyenne** (μ)	**Écart-type** (σ)
x	-0.0136	2.9214
y	-0.0361	2.8851
z	-0.2757	0.7800

Les paramètres sont sauvegardés dans `norm_params.json` pour être réutilisés lors de la dénormalisation des prédictions.

3. Architecture du Réseau MLP

3.1 Structure du réseau

L'architecture choisie est [2, 64, 64, 1] :

Table 3.1: Architecture du MLP

Couche	Type	Neurones	Activation
0	Entrée	2	—
1	Cachée 1	64	ReLU
2	Cachée 2	64	ReLU
3	Sortie	1	Linéaire

3.2 Initialisation des poids — He

L'initialisation de **He** est utilisée pour éviter la disparition du gradient avec ReLU :

$$W \sim \mathcal{N}\left(0,\ \sqrt{\frac{2}{n_{in}}}\right), \quad b = 0 \tag{3.1}$$

```python
def he_initializer(n_in, n_out):
    std = np.sqrt(2.0 / n_in)
    return np.random.randn(n_in, n_out) * std
```

Listing 3.1: Initialisation He — initialisation.py

3.3 Fonctions d'activation

ReLU pour les couches cachées — introduit la non-linéarité :

$$\mathrm{ReLU}(z) = \max(0,\ z) \tag{3.2}$$

Linéaire pour la sortie — adapté à la régression :

$$\mathrm{Linear}(z) = z \tag{3.3}$$

3.4 Fonction de perte — MSE

$$\mathcal{L} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 \tag{3.4}$$

```python
def mse_loss(y_pred, y_true):
    return np.mean((y_pred - y_true) ** 2)
```

Listing 3.2: MSE Loss — loss.py

4. Rétropropagation du Gradient

4.1 Principe

La rétropropagation calcule le gradient de la perte par rapport à chaque paramètre via la **règle de la chaîne** :

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial Z^{(l)}} \cdot \frac{\partial Z^{(l)}}{\partial W^{(l)}} \tag{4.1}$$

4.2 Couche de sortie (activation linéaire)

$$\delta^{(out)} = \frac{2}{m}(\hat{y} - y) \qquad \frac{\partial \mathcal{L}}{\partial W^{(out)}} = \frac{1}{m} A^{(L-1)T} \cdot \delta^{(out)} \tag{4.2}$$

```python
def output_layer_backward(dA, A_prev, W, b, Z, activation='linear'):
    m  = dA.shape[0]
    dZ = dA * 1.0
    dW = (A_prev.T @ dZ) / m
    db = np.sum(dZ, axis=0, keepdims=True) / m
    dA_prev = dZ @ W.T
    return dW, db, dA_prev
```

Listing 4.1: Gradient couche de sortie — gradients_sortie.py

4.3 Couches cachées (activation ReLU)

La dérivée de ReLU est la fonction de Heaviside :

$$\mathrm{ReLU}'(z) = \mathbf{1}[z > 0] \qquad \delta^{(l)} = \left(\delta^{(l+1)} \cdot W^{(l+1)T}\right) \odot \mathbf{1}[Z^{(l)} > 0] \tag{4.3}$$

```python
def relu_derivative(Z):
    return (Z > 0).astype(float)

def hidden_layer_backward(dA_prev, A_prev, W, b, Z, activation='relu'
    ):
    m    = dA_prev.shape[0]
    dZ   = dA_prev * relu_derivative(Z)
    dW   = (A_prev.T @ dZ) / m
    db   = np.sum(dZ, axis=0, keepdims=True) / m
    dA_p = dZ @ W.T
    return dW, db, dA_p
```

Listing 4.2: Gradient couches cachées — gradients_cachees.py

4.4 Mise à jour des poids — SGD

$$W^{(l)} \leftarrow W^{(l)} - \alpha \cdot \frac{\partial \mathcal{L}}{\partial W^{(l)}} \tag{4.4}$$

```python
def sgd_update(weights, biases, grad_weights, grad_biases, learning_rate):
    new_weights = [w - learning_rate * dw for w, dw in zip(weights, grad_weights)]
    new_biases  = [b - learning_rate * db for b, db in zip(biases, grad_biases)]
    return new_weights, new_biases
```

Listing 4.3: Optimiseur SGD — optimizers.py

5. Entraînement et Résultats

5.1 Hyperparamètres

Table 5.1: Hyperparamètres d'entraînement

Hyperparamètre	**Valeur**
Learning rate (α)	0.01
Taille des mini-batchs	64
Nombre d'époques	500
Initialisation	He
Fonction de perte	MSE

5.2 Courbe de loss

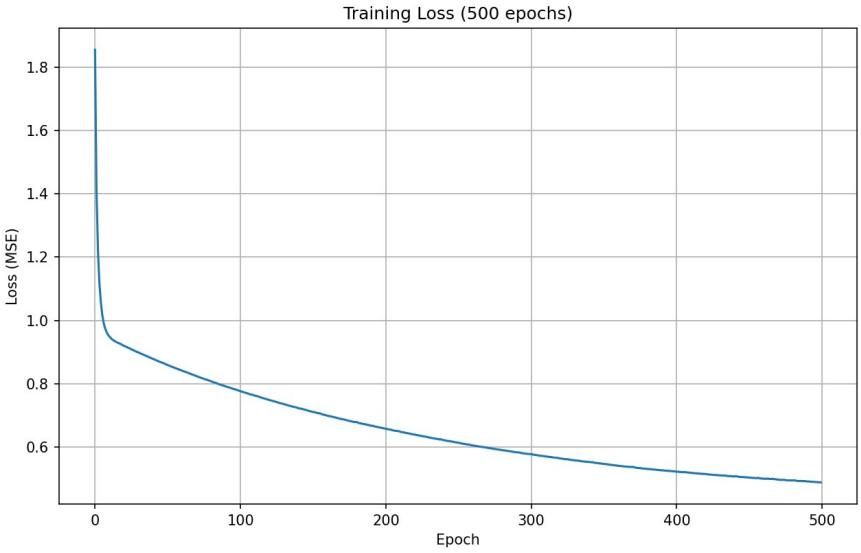


Figure 5.1: Évolution de la MSE durant 500 époques — architecture $[2, 64, 64, 1]$

La loss passe de ≈ 1.85 (epoch 0) à ≈ 0.49 (epoch 500), soit une réduction de **73%**. La descente rapide dans les premières époques confirme l'efficacité de l'initialisation He.

5.3 Comparaison Ground Truth vs Prédictions

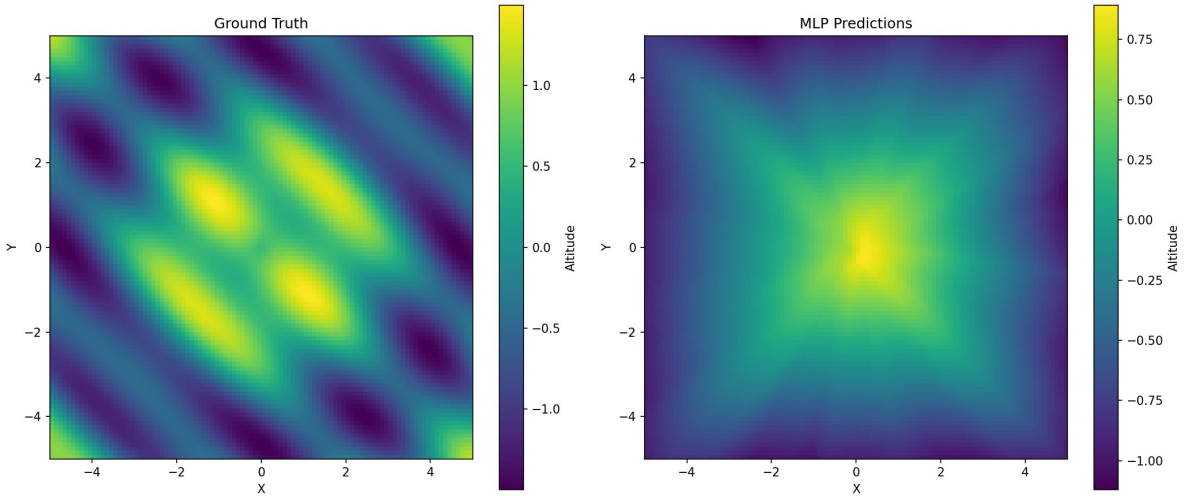


Figure 5.2: Comparaison côte à côte : Surface réelle (gauche) vs Prédictions MLP (droite)

Analyse des résultats

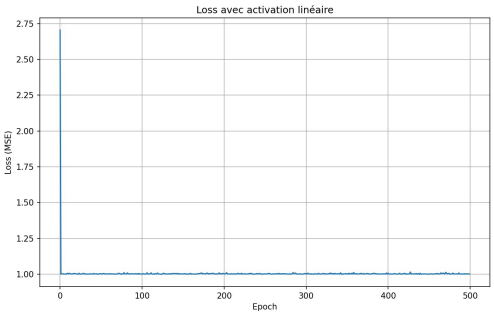
Le MLP capture correctement la structure centrale de la fonction (pic autour de l'origine). Les régions périphériques sont moins précises car la fonction y présente des oscillations haute fréquence difficiles à capturer avec 500 époques seulement. Une augmentation du nombre d'époques ou de neurones améliorerait davantage les prédictions.

6. Questions de Réflexion

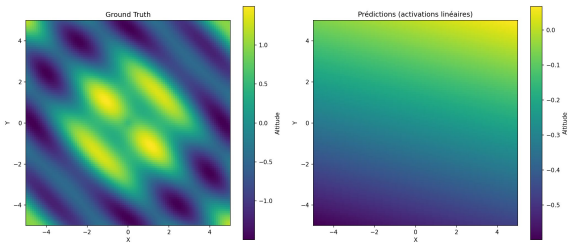
6.1 Q1 — Impact de l'activation linéaire

Expérience

Architecture $[2, 64, 64, 1]$ avec **toutes les activations linéaires** — remplacement de ReLU par Linear dans les couches cachées.



(a) Courbe de loss — activation linéaire



(b) Prédictions — activation linéaire

Figure 6.1: Résultats avec activation linéaire

Analyse : La loss stagne à ≈ 1.0 et ne converge pas. La prédiction est une surface plane sans aucune structure. Ceci s'explique mathématiquement : un MLP entièrement linéaire est équivalent à une régression linéaire simple, quelle que soit sa profondeur :

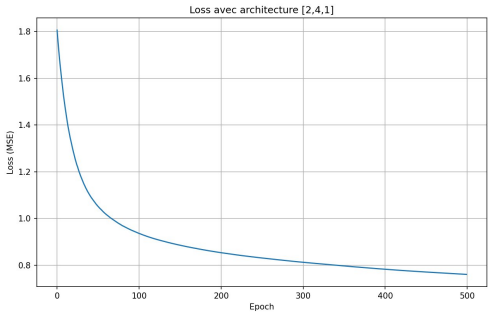
$$f(x) = W_3(W_2(W_1 x + b_1) + b_2) + b_3 = W_{eff} \cdot x + b_{eff} \tag{6.1}$$

Conclusion : Les fonctions d'activation **non-linéaires sont indispensables** pour approximer des fonctions complexes.

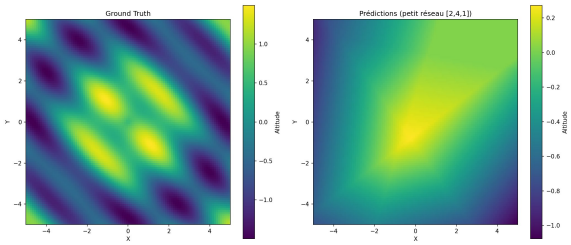
6.2 Q2 — Capacité du réseau (petit réseau)

Expérience

Architecture réduite $[2, 4, 1]$ — seulement **4 neurones** dans la couche cachée.



(a) Courbe de loss — réseau $[2, 4, 1]$



(b) Prédictions — réseau $[2, 4, 1]$

Figure 6.2: Résultats avec le petit réseau $[2, 4, 1]$

Analyse : La loss finale reste à ≈ 0.75, bien supérieure à celle du grand réseau (0.49). Le réseau ne dispose que de $4 \times (2 + 1) + 1 \times (4 + 1) = 17$ paramètres entraînables, insuffisants pour capturer les détails de la fonction. On observe clairement le phénomène de **sous-apprentissage (underfitting)**.

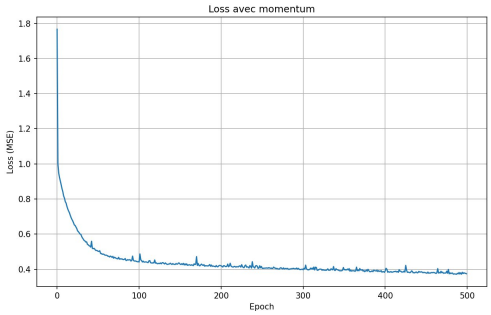
6.3 Q3 — SGD avec Momentum

Expérience

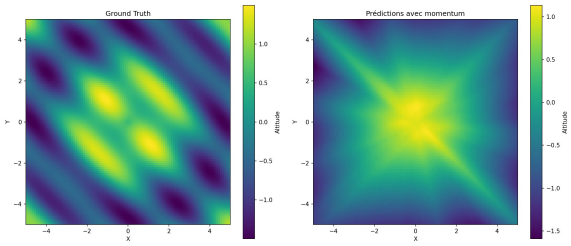
Architecture $[2, 64, 64, 1]$ avec **momentum** $\beta = 0.9$.

Le momentum introduit une inertie dans la mise à jour :

$$v^{(l)} \leftarrow \beta \cdot v^{(l)} - \alpha \cdot \nabla W^{(l)} \qquad W^{(l)} \leftarrow W^{(l)} + v^{(l)} \tag{6.2}$$



(a) Courbe de loss — avec momentum



(b) Prédictions — avec momentum

Figure 6.3: Résultats avec SGD + Momentum ($\beta = 0.9$)

Analyse : Avec momentum, la loss atteint ≈ 0.38 après 500 époques, contre 0.49 avec le SGD classique. La convergence est **plus rapide** mais la courbe présente des **oscillations** caractéristiques du momentum. Les prédictions sont légèrement meilleures.

Comparaison des trois expériences

Configuration	**Loss finale**	**Observation**
SGD standard $[2, 64, 64, 1]$	≈ 0.49	Convergence stable
Activation linéaire $[2, 64, 64, 1]$	≈ 1.00	Ne converge pas
Petit réseau $[2, 4, 1]$	≈ 0.75	Underfitting
SGD + Momentum $[2, 64, 64, 1]$	≈ 0.38	Convergence plus rapide

7. Conclusion

Ce travail pratique a permis d'implémenter *from scratch* un Perceptron Multicouche complet en NumPy, couvrant l'ensemble du pipeline d'apprentissage :

1. **Génération et normalisation** des 2000 points d'entraînement
2. **Conception** d'une architecture MLP flexible et modulaire $[2, 64, 64, 1]$
3. **Rétropropagation vectorisée** sans boucles sur les exemples
4. **Entraînement** par mini-batch SGD sur 500 époques
5. **Analyse comparative** de trois configurations expérimentales

Les résultats confirment que le réseau $[2, 64, 64, 1]$ avec ReLU et initialisation He converge correctement. L'ajout du momentum améliore la vitesse de convergence. Les expériences valident théoriquement l'importance des non-linéarités et de la capacité du réseau.

Annexes

A. Structure du projet

```
ProjetMLP/
|-- SEMAINE1_GENERATION_DATASET/
|    |-- consultation1.py
|    `-- data/  (generate.py, normalize.py, visualize.py)
|-- SEMAINE2_ARCHITECTURE_RESEAU/
|    |-- consultation2.py
|    `-- architecture_reseau/ (mlp.py, activations.py, ...)
|-- SEMAINE3_BACKPROPAGATION/
|    |-- consultation3.py
|    `-- backpropagation/ (train.py, gradients_*.py, optimizers.py)
|-- SEMAINE4_ENTRAINEMENT_ET_VISUALISATION/
|    |-- test_final.py, evaluation.py, visualisation.py
`-- questions_reflexion/
     |-- experiment_linear.py
     |-- experiment_small_network.py
     `-- experiment_momentum.py
```

Listing 7.1: Arborescence du projet

B. Commandes d'exécution

```
# Etape 1 - Generer les donnees
cd SEMAINE1_GENERATION_DATASET && python consultation1.py

# Etape 2 - Tester l'architecture
cd SEMAINE2_ARCHITECTURE_RESEAU && python consultation2.PY

# Etape 3 - Backpropagation
cd SEMAINE3_BACKPROPAGATION && python consultation3.py

# Etape 4 - Test final
cd SEMAINE4_ENTRAINEMENT_ET_VISUALISATION && python test_final.py

# Questions de reflexion
cd questions_reflexion && python experiment_linear.py
cd questions_reflexion && python experiment_small_network.py
cd questions_reflexion && python experiment_momentum.py
```

Listing 7.2: Ordre d'exécution